

AWS AUTOSCALING

AWS Auto Scaling is in high demand today because modern businesses rely on cloud systems that must handle changing workloads efficiently. Applications such as e-commerce, banking, streaming, and enterprise platforms experience unpredictable traffic. Auto Scaling automatically increases or decreases server capacity based on real-time demand, ensuring high performance, availability, and cost optimization.

Companies use Auto Scaling to prevent downtime, improve customer experience, and reduce unnecessary cloud expenses. Since AWS is widely adopted across industries, employers expect cloud and DevOps professionals to understand Auto Scaling as part of production-ready infrastructure. Learning Auto Scaling provides practical, job-relevant skills and improves employability in cloud technology roles.

Task 1: Launch template creation

Step 1: Create a Launch template

Apache-Template (lt-023a8ad357bc348f2) Actions ▾ Delete template

Launch template details

Launch template ID lt-023a8ad357bc348f2	Launch template name Apache-Template	Default version 1	Owner arn:aws:iam::280245841831:root
---	--	-----------------------------	--

Details Versions Template tags

Launch template version details Actions ▾ Delete template version

Version
1 (Default) ▾

Description
-

Date created
2026-06-09T14:38:19.000Z

Created by
arn:aws:iam::280245841831:root

Instance details Storage Resource tags Network interfaces Advanced details

AMI ID ami-0db56f446d44f2f09	Instance type t3.micro	Availability Zone -	Availability Zone Id -
Key pair name junekey	Security groups -	Security group IDs sg-0ea833c9497d4acab	

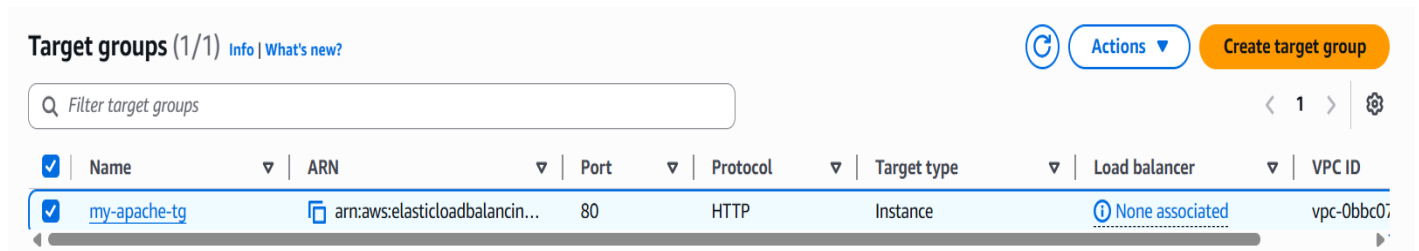
Step 2: Verify that the instance launches successfully and can be accessed using its public IP address.

← → ↺ ⚠ Not secure http://my-app-alb-322477950.ap-south-1.elb.amazonaws.com

Apache Installed Automatically via Launch Template

Task 2: Auto scaling group setup

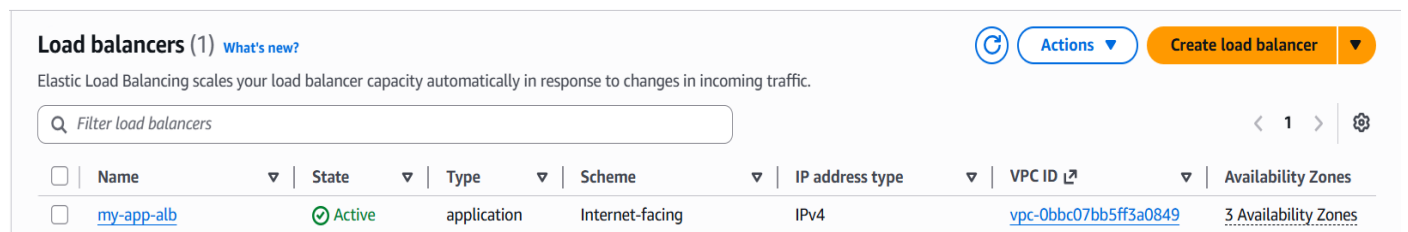
Step 1: Create Target Group



The screenshot shows the AWS Target Groups console. At the top, there's a header with 'Target groups (1/1)', 'Info', 'What's new?', a refresh button, an 'Actions' dropdown, and a 'Create target group' button. Below the header is a search bar labeled 'Filter target groups'. The main content is a table with columns: Name, ARN, Port, Protocol, Target type, Load balancer, and VPC ID. There is one entry in the table:

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
my-apache-tg	arn:aws:elasticloadbalancing...	80	HTTP	Instance	None associated	vpc-0bbc07...

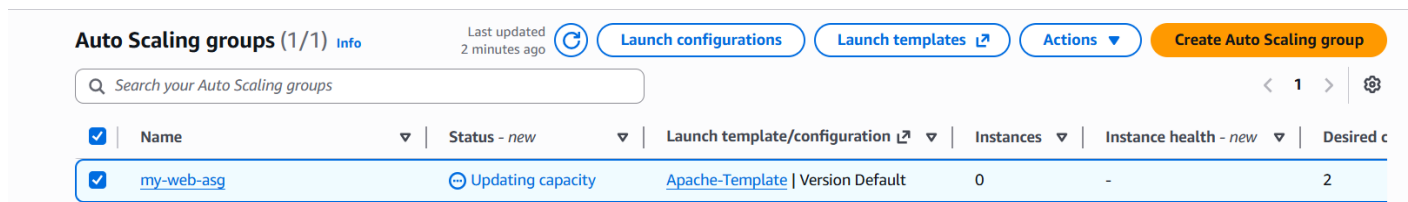
Step 2: Create Load Balancer



The screenshot shows the AWS Load Balancers console. At the top, there's a header with 'Load balancers (1)', 'What's new?', a refresh button, an 'Actions' dropdown, and a 'Create load balancer' button. Below the header is a search bar labeled 'Filter load balancers'. The main content is a table with columns: Name, State, Type, Scheme, IP address type, VPC ID, and Availability Zones. There is one entry in the table:

Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
my-app-alb	Active	application	Internet-facing	IPv4	vpc-0bbc07bb5ff3a0849	3 Availability Zones

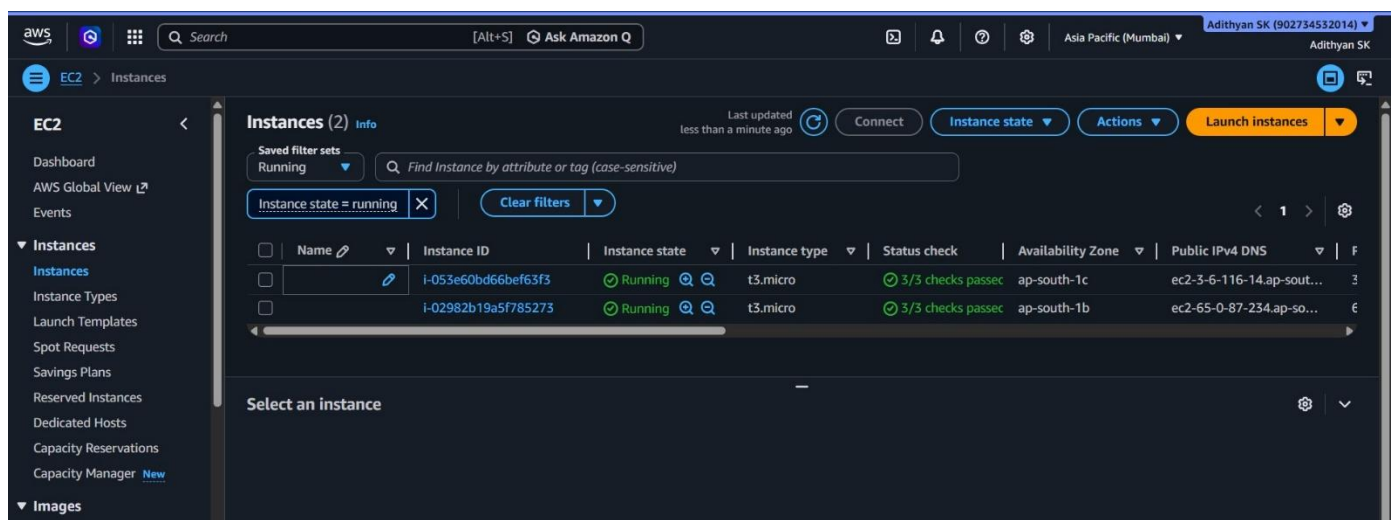
Step 3: Create Auto Scaling Group



The screenshot shows the AWS Auto Scaling Groups console. At the top, there's a header with 'Auto Scaling groups (1/1)', 'Info', 'Last updated 2 minutes ago', a refresh button, 'Launch configurations', 'Launch templates', 'Actions', and 'Create Auto Scaling group' button. Below the header is a search bar labeled 'Search your Auto Scaling groups'. The main content is a table with columns: Name, Status, Launch template/configuration, Instances, Instance health, and Desired capacity. There is one entry in the table:

Name	Status	Launch template/configuration	Instances	Instance health	Desired capacity
my-web-asg	Updating capacity	Apache-Template Version Default	0	-	2

A minimum of 2 instances are launched automatically by the Auto Scaling Group



The screenshot shows the AWS EC2 console. At the top, there's a header with 'EC2', 'Instances', 'Dashboard', 'AWS Global View', 'Events', 'Instances', 'Instance Types', 'Launch Templates', 'Spot Requests', 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', 'Capacity Manager', and 'Images'. The main content is a table with columns: Name, Instance ID, Instance state, Instance type, Status check, Availability Zone, and Public IPv4 DNS. There are two entries in the table:

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
	i-053e60bd66bef63f3	Running	t3.micro	3/3 checks passed	ap-south-1c	ec2-3-6-116-14.ap-sout...
	i-02982b19a5f785273	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-65-0-87-234.ap-so...

Task 4: Scaling Policy Configuration

Configure a CPU-based automatic scaling policy.

Auto Scaling group: my-web-asg

Enabled or disabled

Enabled

Execute policy when

As required to maintain Average CPU utilization at 50

Take the action

Add or remove capacity units as required

Instances need

120 seconds to warm up before including in metric

Scale in

Enabled

Task 5: Load Testing

Step 1: Install the 'stress' package.

```
/m/
[ec2-user@ip-172-31-27-24 ~]$ sudo yum install -y stress
Last metadata expiration check: 0:17:37 ago on Tue Jun  9 14:57:54 2026.
Dependencies resolved.
=====
Package                                Architecture      Version           Repository        Size
-----
Installing:
stress                                x86_64            1.0.7-2.amzn2023.0.1  amazonlinux      34 k
=====
Transaction Summary
-----
Install 1 Package

Total download size: 34 k
Installed size: 68 k
Downloading Packages:
stress-1.0.7-2.amzn2023.0.1.x86_64.rpm 526 kB/s | 34 kB  00:00
-----
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
  Installing     : stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
  Running scriptlet: stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
  Verifying      : stress-1.0.7-2.amzn2023.0.1.x86_64 1/1
```

Step 3: Enter the command `#stress --cpu 4 --timeout 600`

After a few minutes, the Auto Scaling Group scales out and increases the number of instances from 2 to 3.

Instances (3)
Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

< 1 >

⚙

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Put
☐		i-0aad659b8ddf61c80	Running	t3.micro	3/3 checks passed	ap-south-1c	ec2-13-204-156-206.ap...	13.
☐		i-08108ca465d77c339	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-65-2-142-108.ap-s...	65.
☐		i-000f5faa4d895bf1a	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-13-126-149-116.ap...	13.

Task 6: Auto Healing Test

- Step 1: Terminate one instance.
- Step 2: The Auto Scaling Group automatically launches a replacement instance to maintain the desired capacity.

Successful	Launching a new EC2 instance: i-0aed8842987b89168	At 2026-06-09T15:31:29Z an instance was launched in response to an unhealthy instance needing to be replaced.
------------------------------------	---	---

Task 7: Scale In and Cost Optimization

Stop the load-generation command and wait 10–15 minutes. As CPU utilization decreases, the Auto Scaling Group automatically scales in and reduces the number of running instances.

Auto Scaling group: my-web-asg

⚙

⌵

Successful	Terminating EC2 instance: i-043d876b597ab855c	4726-adb8-1be7df8e9fd7 in state ALARM triggered policy cpu-target-tracking-policy changing the desired capacity from 2 to 1. At 2026-06-09T15:11:01Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 2 to 1. At 2026-06-09T15:11:02Z instance i-043d876b597ab855c was selected for termination.	2026 June 09, 08:41:02 PM +05:30	2026 June 09, 08:41:02 PM +05:30
Successful	Launching a new EC2 instance: i-043d876b597ab855c	At 2026-06-09T14:57:19Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-06-09T14:57:24Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 June 09, 08:27:26 PM +05:30	2026 June 09, 08:27:26 PM +05:30
Successful	Launching a new EC2 instance: i-055b808d3ba14aa50	At 2026-06-09T14:57:19Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-06-09T14:57:24Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 June 09, 08:27:25 PM +05:30	2026 June 09, 08:27:25 PM +05:30